

Tokens in Python

Tokens are the smallest units of a Python program. Python has five types of tokens:

1. **Keywords** – Reserved words in Python (e.g., if, else, while, for, def).
2. **Identifiers** – Names given to variables, functions, and classes.
3. **Literals** – Constant values assigned to variables (e.g., 10, "Hello", 3.14).
4. **Operators** – Symbols that perform operations (e.g., +, -, *, /).
5. **Punctuators** – Symbols like (), { }, [], , , : ,. used for syntax structuring.

Sample Program

Python

```
# This program calculates the area of a rectangle

length = 10

width = 5

area = length * width

if area > 30:

    print("Large area")

else:

    print("Small area")
```

Tokens in the Program

Token	Category
#	Comment (not a token, but part of syntax)
length	Identifier
=	Operator

10	Literal
width	Identifier
5	Literal
area	Identifier
*	Operator
if	Keyword
>	Operator
30	Literal
:	Punctuator
print	Identifier (built-in function name)
("Large area")	Punctuators + Literal ((and) are punctuators, "Large area" is a literal)
else	Keyword
"Small area"	Literal

Statements and Identifiers

What is a Statement?

A **statement** in Python is a single line of code that performs an action. It can include:

- Assigning a value to a variable
- Calling a function
- Controlling program flow (like `if`, `for`, or `while`)

Example:

```
Python
x = 10          # Assignment statement
print(x)       # Function call statement
```

What is an Identifier?

An **identifier** is the **name** you give to:

- Variables
- Functions
- Classes
- Modules or objects

Example:

```
Python
age = 25        # 'age' is an identifier for the variable
def greet():    # 'greet' is an identifier for a function
```

Rules for Naming Identifiers

Rule	Description	Example
1	Can include letters (a–z, A–Z), digits (0–9), and underscores (_)	user_name, score1, total_amount
2	Cannot start with a digit	1name is invalid name1 is valid
3	Cannot use Python keywords as names	if = 10 is invalid if_value = 10 is valid
4	Python is case-sensitive	MyVar, myvar, and MYVAR are all different identifiers
5	Avoid using special characters like @, #, \$, etc.	user@name is invalid

Examples of Valid and Invalid Identifiers

Valid Identifiers	Invalid Identifiers	Reason
name	1st_place	Starts with a digit
user_123	class	class is a Python keyword
_temp	user-name	Contains invalid character -
totalAmount	@value	Contains invalid character @

Sample Code

Python

```
age = 25          # 'age' is an identifier (valid)

_name = "John"    # '_name' is a valid identifier

score1 = 88       # valid: letters + digits


# Invalid examples (will cause errors if uncommented)

# 1age = 30       # Invalid: starts with a digit

# def = 5         # Invalid: 'def' is a Python keyword
```

Comments in Python

Comments are used to explain code and are ignored by the Python interpreter.

Types of Comments

Single-line Comment (#)

Python

```
# This is a single-line comment

x = 10 # This is also a comment
```

Multi-line Comment (''' or ''')

Python

```
''' This is a multi-line comment.

It spans multiple lines.

'''

print("Hello, World!")
```

What Are Keywords?

Keywords are **reserved words** in Python that have special meanings and purposes. They are part of the Python syntax and **cannot be used as identifiers** (variable, function, or class names).

How to Access All Keywords in Python

You can view the full list of Python keywords using the built-in keyword module:

```
Python
import keyword

print(keyword.kwlist)      # List of all keywords
print(len(keyword.kwlist)) # Total number of keywords
```

List of Python Keywords

Keyword	Description
False	Boolean false value
None	Represents a null value
True	Boolean true value
and	Logical AND operator
as	Used to create an alias

<code>assert</code>	Debugging aid that tests a condition
<code>async</code>	Declares an asynchronous function
<code>await</code>	Waits for the completion of an async function
<code>break</code>	Exits a loop prematurely
<code>class</code>	Defines a class
<code>continue</code>	Skips the rest of the loop and starts next iteration
<code>def</code>	Defines a function
<code>del</code>	Deletes an object or variable
<code>elif</code>	Else if condition in an if statement
<code>else</code>	Executes when if or elif conditions are false
<code>except</code>	Catches exceptions in a try block
<code>finally</code>	Block that always runs after try/except
<code>for</code>	Looping over a sequence

from	Imports specific parts of a module
global	Declares a global variable
if	Starts a conditional block
import	Imports a module
in	Checks for membership in a sequence
is	Checks for object identity
lambda	Defines an anonymous function
nonlocal	Refers to a variable in the nearest enclosing scope
not	Logical NOT operator
or	Logical OR operator
pass	Does nothing; a placeholder
raise	Raises an exception
return	Exits a function and optionally returns a value

try	Starts a block to catch exceptions
while	Starts a loop while a condition is true
with	Wraps code block within context managers
yield	Returns a generator from a function
match	Starts a pattern-matching block (Python 3.10+)
case	Defines a case in a match statement (Python 3.10+)

Variables in Python

What is a Variable?

A **variable** is a **named memory location** used to store data in a Python program. It acts as a label for a value that you want to use or modify during program execution.

Variable Naming Rules

Rule	Description	Example
1	Must start with a letter (A–Z, a–z) or underscore	name, _value
2	Cannot start with a number	1stvalue is invalid
3	Can contain letters, digits, and underscores	user_name1 is valid
4	Python is case-sensitive	age, Age, AGE are different

Basic Variable Assignment

```
Python
product_name = "Laptop"
price = 45000
in_stock = True

print(product_name, price, in_stock)
```

Multiple Assignment

Python allows assigning values to multiple variables in a single line.

```
Python
a, b, c = 10, 20, 30
print(a, b, c) # Output: 10 20 30
```

You can also assign the **same value** to multiple variables:

```
Python
x = y = z = 100
print(x, y, z) # Output: 100 100 100
```

Reassignment

A variable's value can be updated or reassigned at any point.

```
Python
x = 5
x = 10
print(x) # Output: 10
```

Swapping Variables

Python allows swapping values of two variables **without using a temporary variable**:

Python

```
a, b = 5, 10
a, b = b, a
print(a, b)  # Output: 10 5
```

Deleting Variables

Use the `del` keyword to delete a variable:

Python

```
x = 100
del x

# print(x)  # Raises: NameError: name 'x' is not defined
```